

브랜치 관리 한눈에 보기

개발자뿐 아니라 디자이너·기획자도
Git/PR 흐름을 그림으로 이해할 수 있는
1주차 온보딩 가이드

대상	Git/PR을 처음 다루는 신규 입사자
목표	"어느 라인에서 출발해 어디로 합쳐지는가"의 흐름 이해
읽기 시간	약 15분 (5분 요약 → 1분)
태그	Branch PR Cascade Promotion

🎬 5분 요약 — 핵심 비유

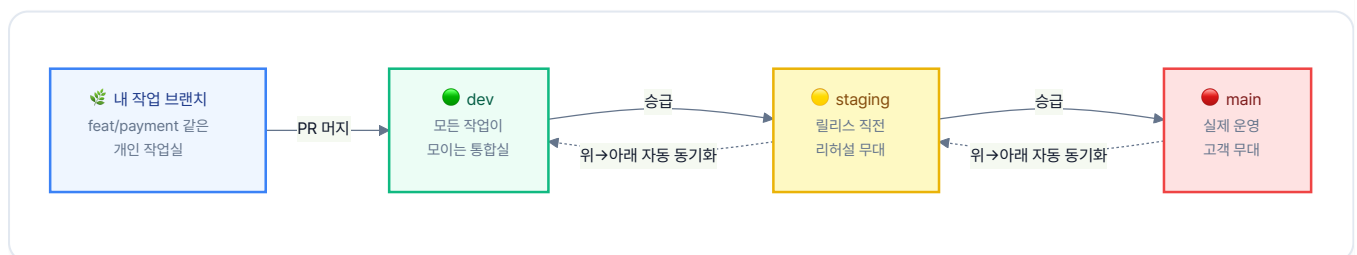
코드가 모이는 라인을 공연장 무대라고 생각해 보세요.

라인	비유	설명
🔴 main	본 공연 무대	실제 고객이 보는 우리 서비스. 사고가 나면 안 됩니다
🟡 staging	리허설 무대	본 공연 직전 마지막 점검. "이 상태로 무대에 올려도 될까?"
🟢 dev	연습실	모든 새 작업이 처음 모이는 곳. 자유롭게 합치고 깨고 다시 만듭니다
🌿 feat/fix/...	개인 연습실	각자가 자기 작품을 만드는 공간. 외부에는 보이지 않음

작업 흐름: 개인 연습실에서 만든 작품을 → 연습실(dev)에서 합쳐 → 리허설(staging)에서 점검 → 본 공연(main)에 올린다.

01

전체 흐름 — 큰 그림



개인 작업실 → dev(연습실) → staging(리허설) → main(본 공연)

읽는 법

- **실선 화살표(→)**는 사람이 시작하는 흐름: PR을 만들고, 검증 후 다음 단계로 "승급"시킵니다.
- **점선 화살표(↗)**는 도구가 자동으로 처리하는 흐름: 위 라인에 변경이 생기면, 아래 라인들로 즉시 동기화됩니다.

라인	누가 손대나?	어떻게 바뀌나?
개인 작업 브랜치	본인	자유롭게 저장(commit)
dev	누구도 직접 손대지 않음	작업 브랜치의 PR(합치기 요청) 머지로만
staging	릴리스 담당자	dev → staging 승급(promotion)으로만
main	릴리스 담당자	staging → main 승급으로만

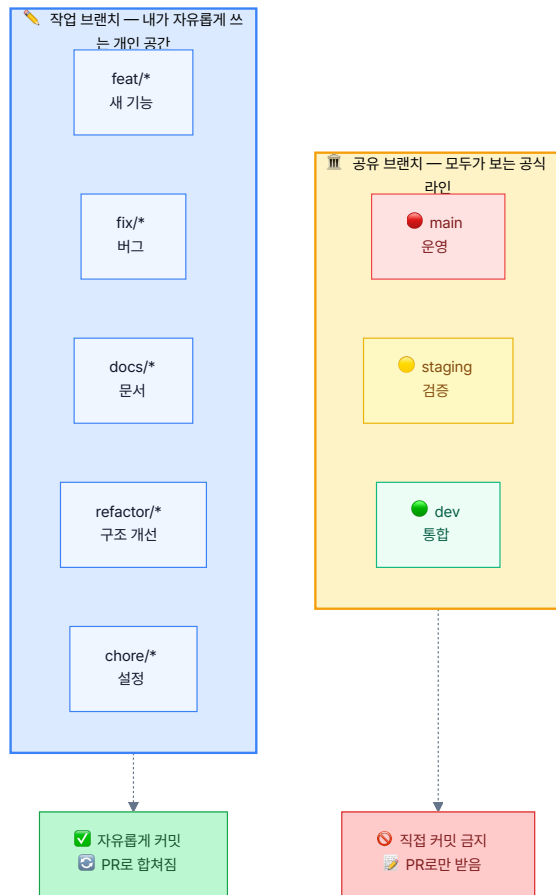
💡 핵심

dev / staging / main은 공식 라인입니다. 절대 직접 저장하지 않고, 반드시 PR이나 승급을 거쳐 들어옵니다.

02

공유 브랜치 vs 작업 브랜치

브랜치는 크게 두 가지 성격으로 나뉩니다.



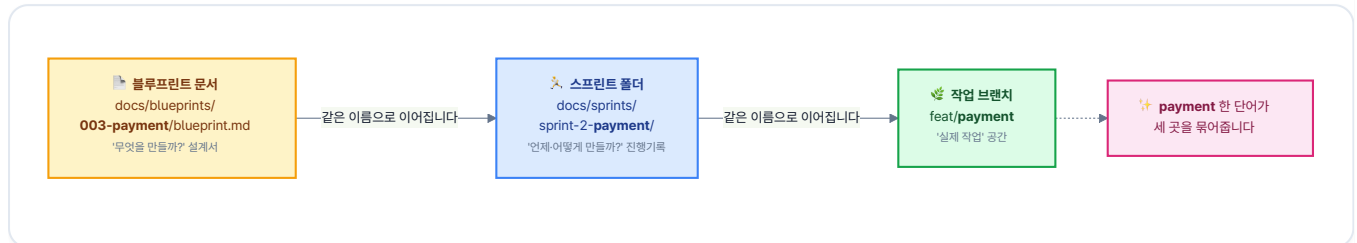
구분	어떤 브랜치	직접 저장 가능?	언제 만들어지나?
공유 브랜치	main, staging, dev	❌ PR로만 변경 가능	저장소를 처음 만들 때 한 번
작업 브랜치	feat/*, fix/*, docs/*, refactor/*, chore/*	✅ 본인이 자유롭게 저장	새 작업을 시작할 때마다

왜 이렇게 나누나?

- 공유 브랜치를 직접 건드리면 다른 동료가 보고 있던 상태가 갑자기 바뀝니다. 검토 없이 들어간 변경은 사고의 원인.
- 작업 브랜치는 본인만의 공간이라 망쳐도 부담 없이 다시 시작할 수 있습니다.

브랜치 이름은 어떻게 정해지나

ASTRA는 세 가지 산출물의 이름을 한 단어로 묶습니다. PR을 봤을 때 "어느 기획에서 시작된 어느 스프린트의 작업인지" 즉시 추적되도록.



작명 규칙 4가지

자리	규칙	예시
{feature-name}	영문 소문자 + 하이픈만 (kebab-case). 한국어·대문자·언더스코어 ❌	payment, user-auth, checkout-flow
{NNN}	블루프린트 번호. 3자리 (001, 002, ...)	003-payment
{N}	스프린트 순번. 0 채우기 없음 (1, 2, 3, ...)	sprint-2-payment
{prefix}	변경의 성격 (아래 표)	feat/payment

프리픽스 5종

프리픽스	언제 쓰나	예시
feat/	새 기능 추가 (블루프린트 기반 작업의 기본값)	feat/payment
fix/	버그 수정	fix/login-error, fix/checkout-crash
docs/	문서만 변경 (코드 미수정)	docs/onboarding-guide
refactor/	코드 구조 정리 (동작은 그대로)	refactor/payment-module
chore/	빌드·설정·도구 (서비스 동작과 무관)	chore/eslint-bump

한 줄 추적 예시

docs/blueprints/003-payment/blueprint.md (기획)
→ docs/sprints/sprint-2-payment/ (스프린트 진행 기록)

→ 🌿 feat/payment (작업 브랜치)

→ 📄 PR 제목: feat: 결제 기능 추가

💡 자동 결정

작업 브랜치 이름은 표준 도구가 변경 성격을 분석해 자동으로 만듭니다. 직접 외울 필요 없음 — 다만 **블루프린트**와 같은 **{feature-name}** 을 쓴다는 원칙만 기억하세요.

04

한 저장소에서 여러 작업을 동시에 — 작업 공간 격리

I 왜 격리가 필요한가?

한 컴퓨터의 같은 저장소에서 **여러 개의 Claude Code 세션**을 동시에 띄워 서로 다른 작업을 진행하고 싶을 때가 있습니다 (예: 결제 기능 + 로그인 버그 + 온보딩 문서 작업을 병행).

평범한 방식(`git checkout`)으로는 이게 불가능합니다.

- 세션 A가 `feat/payment` 로 전환하는 순간 → 세션 B가 보고 있던 파일이 모두 다른 내용으로 바뀝니다.
- 세션 B는 자기 작업이 사라진 줄 알고 놀라게 됩니다.

■ 해결: 브랜치마다 독립된 폴더



각 작업 브랜치는 **저장소 안의 별도 폴더**(git worktree)에서 진행됩니다. 폴더가 다르니 파일도 독립적입니다.

■ 핵심 동작 4가지

1. **메인 작업 공간은 공유 브랜치만** — 저장소 루트는 항상 `dev` / `staging` / `main` 중 하나만 유지. 승급·동기화는 여기서.
2. **폴더 이름 규칙** — 브랜치명의 `/` 를 `-` 로: `feat/payment` → `.astra-worktrees/feat-payment/`
3. **자동 생성·자동 정리** — 작업 브랜치를 만들 때 폴더도 함께 생김. PR 머지가 끝나면 자동 삭제.
4. **중단되면 그대로 남음** — 충돌·검토 대기로 멈추면 폴더는 사라지지 않음. 나중에 그 자리에서 이어서.

💡 결과

한 사람이 Claude 세션 두 세 개를 동시에 띄워 결제·로그인·문서 작업을 함께 진행해도 충돌 없이 병행됩니다.

캐스케이드 동기화 — 매번 일어나는 안전 장치

어떤 일이 일어나는가?

PR을 만들기 직전, 도구가 다음 순서대로 위에서 아래로 변경을 흘립니다.



왜 매번 이렇게 하나?

내 `feat/login` 브랜치가 일주일 전 `dev` 에서 갈라져 나왔다면, 그 사이에 `dev` 에는 다른 동료의 코드가 잔뜩 들어왔을 것입니다. 그 상태로 PR을 올리면:

- 충돌 폭발 — 리뷰 흐름이 끊기고 시간이 낭비됨
- CI 통과 → 머지 후 깨짐 — 옛날 `dev` 기준으로 검증돼서 신뢰 못 함
- 운영 핫픽스 역행 — `main`에 들어간 수정이 `dev`에 반영 안 되면, 다음 릴리스에서 같은 버그가 부활

⚙️ 캐스케이드의 역할

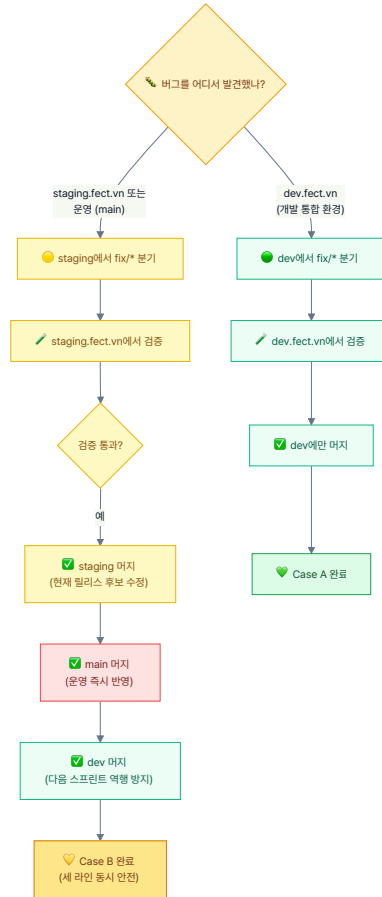
"위 라인의 모든 변경이 항상 아래 라인에 있다"는 규칙을 **매 PR마다 강제**합니다. 사람이 잊지 않게 도구가 매번 합니다.

충돌이 나면?

자동으로 해결하지 않습니다 — 반드시 작업자가 직접 해결합니다. 코드의 의도는 사람만 알 수 있기 때문입니다.

버그픽스가 까다로운 이유 — 출발점이 다르면 길도 다르다

같은 `fix/*` 브랜치라도 어디서 발견했는가에 따라 머지 경로가 완전히 달라집니다.



📌 한 줄 규칙

버그를 본 환경의 브랜치에서 출발하고, 같은 환경에서 검증한다. 이래야 "내가 본 그 상태"에서 정확히 고친다는 보장이 생깁니다.

Case A — dev 환경 버그 (일반)

단계	내용
어디서 발견?	<code>dev.fect.vn</code> (개발 통합 환경)
어디서 출발?	<code>dev</code> 브랜치
어디서 검증?	<code>dev.fect.vn</code>
어디로 합치나?	<code>dev</code> (한 곳만)

평소 기능 개발과 동일한 흐름입니다.

Case B — staging/운영 버그 (릴리스 직전·직후)

단계	내용
어디서 발견?	<code>staging.fect.vn</code> 또는 운영(main)
어디서 출발?	<code>staging</code> 브랜치
어디서 검증?	<code>staging.fect.vn</code>
어디로 합치나?	<code>staging</code> + <code>main</code> + <code>dev</code> 세 곳 모두

왜 Case B는 세 곳에 다 합쳐야 하나?

`staging` 은 곧 `main` 이 됩니다. 그래서 `staging` 에만 고치면:

빠뜨린 곳	결과
<code>main</code> 에 안 가면	운영은 여전히 버그 상태 (고객 피해 지속)
<code>dev</code> 에 안 가면	다음 스프린트에서 새 코드가 흐르며 같은 버그가 부활 (역행)

⚠ 반드시 기억

Case B = "고친 곳 + 위(main) + 아래(dev)" 세 방향 **동시 반영**. 빠뜨리면 며칠 뒤 같은 버그가 다시 올라옵니다.

절대 하지 말 것

- ❌ `staging` 에서 봤는데 `dev` 에서 출발 — 환경 차이로 재현 안 될 수 있음

- **✗** Case B에서 `staging`에만 머지 — 위 "역행" 발생
- **✗** Case B를 Case A처럼 `dev`만 거쳐 보내기 — `staging`→`main` 라인이 그동안 노출 상태

07

프로모션(승급) — 누가, 언제, 무엇을 확인하나

⚠ 회사 고유 규정

아래 표는 팀에서 직접 채워 넣어야 합니다. 본 공연 무대에 올리는 결정이니 책임자와 시점이 명확해야 합니다.

전이	누가 진행?	언제?	사전 확인
<code>dev</code> → <code>staging</code>	TODO (예: 스프린트 리더)	TODO (예: 스프린트 종료 목요일 17시)	TODO (예: dev E2E 통과, QA 사인오프)
<code>staging</code> → <code>main</code>	TODO (예: 릴리스 매니저)	TODO (예: 격주 화요일 10시)	TODO (예: staging 48시간 무이슈, 릴리스 노트 작성)

버전 번호 올리기 (SemVer)

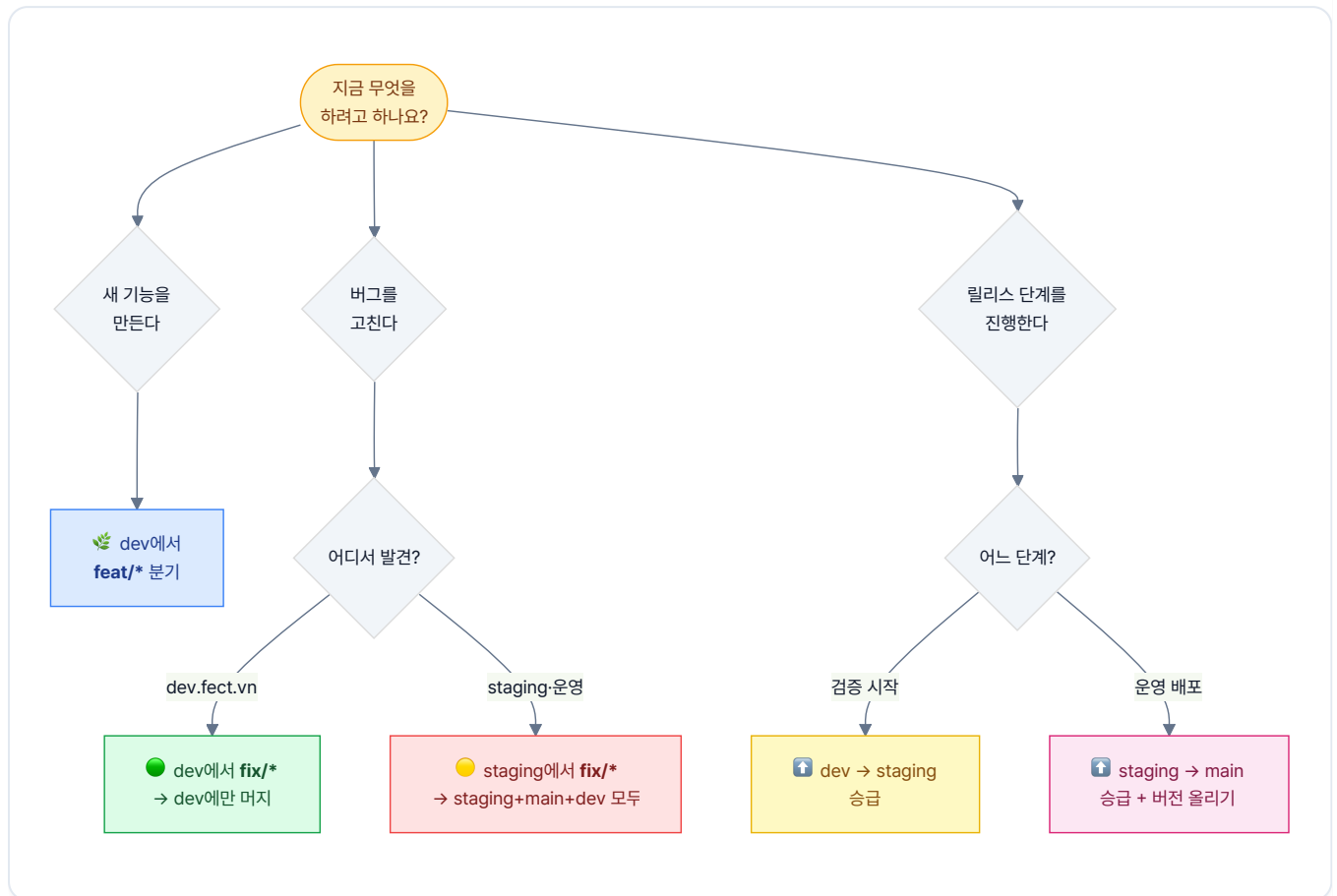
`staging` → `main` 승급 시 릴리스 버전이 한 칸 올라갑니다. 마치 잡지 호수가 1호 → 2호로 올라가듯이.

종류	변화	언제
<code>patch</code>	<code>1.2.3</code> → <code>1.2.4</code>	버그 수정만 있을 때
<code>minor</code>	<code>1.2.3</code> → <code>1.3.0</code>	새 기능이 추가될 때
<code>major</code>	<code>1.2.3</code> → <code>2.0.0</code>	호환성이 깨질 때 (드물게)

판단 기준: 기존 사용자의 코드가 깨지면 `major`, 새 기능이면 `minor`, 그 외는 `patch`.

08

한 장의 의사결정 트리



복잡해 보여도 결국 세 갈래입니다. 새 기능 / 버그 수정 / 릴리스 단계.

09

자주 묻는 질문 (FAQ)

Q. 작업하다가 다른 일이 끼어들면 어떻게 하나요?

지금 작업 브랜치는 그대로 두고, 새 작업 브랜치를 따로 만들어 진행합니다. 작업 공간이 격리되어 있어 영향이 없습니다 (§4 참고).

Q. PR 리뷰에서 Critical 이슈가 나오면?

Critical 이슈가 1건이라도 남으면 머지가 차단됩니다. 수정 후 같은 PR을 갱신합니다.

Q. `git push --force` 같은 강제 명령을 써도 되나요?

공유 브랜치(`main` / `staging` / `dev`)에는 **절대 금지**. 작업 브랜치는 본인 PR에 한해 가능하지만, 일반적으로 필요 없습니다.

Q. 어디서 출발해야 할지 헷갈리면?

버그를 본 환경의 브랜치에서 출발한다는 규칙을 외우세요. `dev.fect.vn` 에서 봤으면 `dev` , `staging.fect.vn` 또는 운영에서 봤으면 `staging` .

Q. 캐스케이드 충돌이 너무 자주 납니다.

작업 브랜치 수명을 짧게 가져가세요. 1주 이상 안 합치는 브랜치는 위험합니다. **작게 자주 합치는 것**이 정답입니다.

Q. 디자이너/기획자도 브랜치를 만들어야 하나요?

변경하는 산출물(디자인 토큰, 기획 문서)이 저장소에 들어간다면 **네**. 다만 도구가 브랜치 생성을 대행해주므로 명령어를 외울 필요는 없습니다. **흐름만** 알고 있으면 됩니다.

10

입사 첫 주 체크리스트

- ☐ **1일차** — 이 문서를 정독, 책상 옆에 1장 인쇄해 두기
- ☐ **2~3일차** — 첫 PR은 멘토와 함께. **어디서 출발했고 어디로 합치는지에 집중**
- ☐ **4~5일차** — `dev → staging` 승급 과정을 옆에서 관찰
- ☐ **2주차** — 첫 `staging` 버그가 생기면 멘토와 함께 §6 Case B 실습
- ☐ **계속** — 막힐 때마다 이 문서로 돌아오기. **외울 것은 명령어가 아니라 흐름**

Last updated: TODO (날짜 채우기) · Maintainer: TODO (담당자 채우기)

ASTRA Methodology · Branch Strategy Training v1.0